



BitFlip

CloudSherpa

Coding Standards Document

Contents

1	Introduction	2
2	Repository & File Structure	2
3	Naming Conventions	2
3.1	Variables and Functions	2
4	Formatting, Configurations, and Style	3
4.1	Prettier	3
4.2	Spotless	5
5	Code Quality Best Practices	8
5.1	Static Code Analysis	8

1 | Introduction

2 | Repository & File Structure

Directory Structure

```
CloudSherpa/  
  apps/                # Monorepo application source code  
    dashboard/         # Frontend web application  
    ingestion/         # Cloud data integration service  
    intelligence/       # Analytics and forecasting service  
    lib/               # Shared internal libraries  
    service/           # Core backend API services  
  design-system/       # Design system as separate app  
  docs/                # Project documentation  
  infra/               # Deployment configurations  
  persistence/         # Database schemas (sherpadb) and .env configs  
  scripts/             # Local dev helpers (hooks, env-init, mocks)  
  pom.xml              # Root Maven configuration  
  sonar-project.properties # SonarCloud code quality configuration
```

3 | Naming Conventions

3.1 Variables and Functions

Standard Rule

Use **camelCase** for all local variables, instance variables, and function/method names. Use **UPPER_SNAKE_CASE** exclusively for global constants (e.g., static final fields).

✔ Preferred Standard

```
1 public class ProviderConfig {
2     public static final int MAX_RETRY_ATTEMPTS = 5;
3
4     private int connectionTimeout;
5
6     public void normalizeIngestedData() {
7         // Implementation details
8     }
9 }
```

4 | Formatting, Configurations, and Style

The CloudSherpa project uses automated code formatters to maintain consistent formatting and coding style across its applications. The formatters and their respective configurations are described below.

4.1 Prettier

Prettier is used to enforce consistent formatting throughout the dashboard application, which is written in TypeScript and JSX (JavaScript Syntax Extension). It is configured as follows:

apps/dashboard/.prettierrc

```
{
  "singleQuote": false,
  "printWidth": 100,
  "tabWidth": 4,
  "semi": true,
  "trailingComma": "es5",
```

```

    "bracketSpacing": true,
    "endOfLine": "lf"
  }

```

The configuration options have the following effects:

- `"singleQuote": false` uses double quotation marks for strings.
- `"printWidth": 100` sets the preferred maximum line width to 100 characters.
- `"tabWidth": 4` represents each indentation level using four spaces.
- `"semi": true` adds semicolons at the end of statements.
- `"trailingComma": "es5"` adds trailing commas where supported by the ECMAScript 5 convention.
- `"bracketSpacing": true` adds spaces inside object literal braces.
- `"endOfLine": "lf"` standardises line endings using the line feed (LF) convention rather than carriage return and line feed (CRLF).

Developers can automatically format the dashboard code by running the `npm run format:fix` script. This allows developers to work using their preferred formatting style while ensuring that the committed code conforms to the project's standard.

A pre-commit hook enforces this standard by running `npm run format:check` before each commit. If the check fails, the commit is rejected and the developer is instructed to run `npm run format:fix` before attempting to commit again.

✔ Preferred Standard

```

1 import { cn } from "@lib/utils";
2 import { Loader2Icon } from "lucide-react";
3
4 function Spinner({ className, ...props }: Readonly<React.
  ComponentProps<"svg">>) {
5   return (
6     <Loader2Icon
7       role="status"
8       aria-label="Loading"
9       className={cn("size-4 animate-spin", className)}
10      {...props}
11    />
12   );
13 }
14
15 export { Spinner };

```

4.2 Spotless

The Ingestion and Service applications are written in Java. The Spotless code formatter is configured to enforce a customised version of the Google Java Style Guide, summarised as follows:

- Use one top-level class per source file, with the filename matching the class name.
- Structure source files in the following order: licence header, package declaration, imports, and class declaration.
- Do not use wildcard imports. Separate static and non-static imports and sort them alphabetically.
- Use braces for all control structures, including single-line statements.
- Place opening braces on the same line as the corresponding declaration or statement.
- Use two spaces for indentation and do not use tabs.
- Limit lines to 100 characters, except where defined exceptions apply.
- Place each statement on its own line.
- Break long expressions at logical points and indent continuation lines by at least four spaces.
- Use spaces around operators and after commas, and avoid unnecessary horizontal alignment.
- Use UpperCamelCase for classes and interfaces.
- Use lowerCamelCase for methods, variables, and parameters.
- Use UPPER_SNAKE_CASE for constants.
- Use lowercase package names without underscores.
- Declare one variable per declaration and keep variables close to where they are first used.
- Indent switch cases and include a default case where appropriate.
- Place annotations on separate lines, except where inline annotations improve readability.
- Use comments to explain intent or non-obvious decisions rather than restating the code.

- Use the `@Override` annotation whenever a method overrides or implements another method.
- Do not silently ignore caught exceptions; provide a clear justification when an exception is intentionally suppressed.
- Apply the style consistently rather than relying on individual formatting preferences.

The Google Java Style Guide recommends using Javadoc for public APIs where documentation is required. This requirement has been relaxed because the project's public APIs are documented using the OpenAPI standard.

✓ Preferred Standard

```
1 package com.cloudsherpa.ingestion.models;
2
3 import java.util.Collections;
4 import java.util.Map;
5
6 public class ResourceDetail {
7
8     private final String resourceId;
9     private final String name;
10    private final String resourceType;
11    private final String serviceCategory;
12    private final Map<String, String> tags;
13
14    public ResourceDetail(
15        String resourceId,
16        String name,
17        String resourceType,
18        String serviceCategory,
19        Map<String, String> tags) {
20        this.resourceId = resourceId;
21        this.name = name;
22        this.resourceType = resourceType;
23        this.serviceCategory = serviceCategory;
24        this.tags = tags == null ? Collections.emptyMap() : tags;
25    }
26
27    public String getResourceId() {
28        return resourceId;
29    }
30
```

```
31 public String getName() {
32     return name;
33 }
34
35 public String getResourceType() {
36     return resourceType;
37 }
38
39 public String getServiceCategory() {
40     return serviceCategory;
41 }
42
43 public Map<String, String> getTags() {
44     return tags;
45 }
46
47 public static String resolveName(
48     String resourceId, String explicitName, Map<String, String>
49     tags) {
50
51     if (explicitName != null && !explicitName.isBlank()) {
52         return explicitName;
53     }
54
55     if (tags != null) {
56         String tagName = tags.get("Name");
57         if (tagName != null && !tagName.isBlank()) {
58             return tagName;
59         }
60     }
61
62     return resourceId;
63 }
```


5 | Code Quality Best Practices

5.1 Static Code Analysis

To improve code quality, maintainability, performance, and the detection of common security vulnerabilities, the CloudSherpa project uses static code analysis provided by the Sonar suite. In addition to the SonarCloud analysis performed for each pull request, Team BitFlip hosts a dedicated SonarQube Server instance.

SonarQube is integrated into the development workflow as follows:

- Developers can run static analysis on demand using the Sonar CLI, allowing them to identify issues before committing their changes.
- A separate SonarQube project is maintained for each application. The relevant analysis is triggered by the CI workflows according to which applications were modified.
- Custom quality gates are used to enforce stricter standards. For the Ingestion and Service applications, the quality gate fails whenever a new issue is introduced. This is stricter than the default SonarCloud rule, which permits new issues below a specified threshold. When a quality gate fails, the corresponding CI check also fails.
- SonarQube issues can be assigned to individual team members, promoting accountability and ownership of code quality.